Amstrad CPC · Z80

Manual de uso de GlasnostFWK

Construcción modular de juegos y pruebas técnicas con CPCTelera Next

Edición: 4 de noviembre de 2026 · Idioma: castellano

1. Qué es y qué cubre este manual

GlasnostFWK es un framework modular en ensamblador Z80 para Amstrad CPC. Un proyecto selecciona únicamente los módulos que necesita mediante un manifiesto JSON; el constructor prepara un árbol aislado y CPCTelera Next genera los medios CPC.

Este manual describe el estado ejecutable comprobado del repositorio: núcleo de sistema, entrada, entity manager ECS, vídeo Mode 0, sprites rectangulares con clipping, restauración, orden explícito y máscaras opcionales, texto, ventanas restaurables, movimiento ECS, perfiles separados de plataforma y física 8.8, consulta, colisión y render de tilemaps, animación, audio AKG, bancos de 128 KB, carga AMSDOS y ZX7B. La rama `ecs-v2` contiene la ABI ECS normativa, el manager ejecutable y la generación validada de perfiles ECS desde el manifiesto.

> **Límites importantes.** No hay verificación en hardware real ni garantía temporal global de 50 Hz. El perfil 160×288 tiene comprobado su modelo de memoria, pero su señal CRTC continúa siendo experimental. La física 8.8 no incluye colisión entre cuerpos y el tilemap continúa siendo de solo lectura. El dibujo enmascarado 6x8 tiene coste medido, pero no se ha integrado en el lote ordenado.

Índice

1. Qué es y qué cubre
2. Requisitos
3. Primeros pasos
4. Estructura del repositorio
5. Crear un proyecto o ejemplo
6. Referencia de módulos
7. ABI y entity manager ECS 2.0
8. Perfiles y recursos
9. Ejemplos incluidos
10. Pruebas y emulación
11. Distribución
12. Diagnóstico
13. Mantener este manual

2. Requisitos

- Python 3 y GNU Make.
- CPCTelera Next en la revisión fijada `11e61fe30175ad7b328de9bcd859c46fd61b26a5`, con SDCC Z80 4.5.0 y sus herramientas construidas.
- Ruta predeterminada: `/config/workspace/cpctelera_next/cpctelera`. Puede sustituirse con `CPCT_NEXT_PATH` o con `--cpct-path`.
- AmSpiriT-Lite 1.15.3 solo para ejecutar la batería de emulador existente.
- WeasyPrint 70.0 solo para regenerar este manual en PDF.

```
export CPCT_NEXT_PATH=/ruta/a/cpctelera
python3 tools/build_examples.py --cpct-path "$CPCT_NEXT_PATH" minimal
```

La compatibilidad con instalaciones distintas de la revisión fijada no está confirmada. Si se cambia, hay que volver a compilar y ejecutar las pruebas.

3. Primeros pasos

3.1 Construir todo

```
cd /config/workspace/GlasnostFWK
make generate
make check
make examples
make test
```

`make generate` produce las tablas de vídeo y recursos. Durante el desarrollo, `make check` ejecuta toda la suite host y compila solo `platformer`; puede seleccionar otros consumidores con `make check CHECK_EXAMPLES="ecsmanager platformer"`. No reconstruye el DSK común. `make examples` construye los veintidós consumidores y crea el disco de pruebas en `.build/tests/GlasnostFWK-tests.dsk`. `make test` combina suite host, construcción completa y DSK y se reserva para cerrar una tarea o preparar un commit.

> **Política de validación por niveles.** En cada iteración use pruebas focalizadas o `make check`. Ejecute un único smoke CPC 6128 solo si cambió código Z80 ejecutado. Repita en CPC 464 únicamente si el cambio depende del modelo, memoria o bancos. Regenere PDF una vez, cuando el texto esté cerrado. Antes del commit ejecute `make test` una sola vez; las releases sí requieren la matriz completa aplicable.

3.2 Construir y ejecutar un único ejemplo

Cada ejemplo es un punto de entrada independiente, igual que un proyecto CPCtelera. Desde su propia carpeta:

```
cd examples/minimal
make        # compila este ejemplo y copia BIN, CDT y DSK a la carpeta
make run    # compila y lo abre en el emulador
make clean  # borra .build/minimal/ y las copias locales
```

`make` reutiliza `tools/build_examples.py` y deja el proyecto generado en `.build/<nombre>/`. Además copia junto a las fuentes `examples/<nombre>/<nombre>.bin`, `.cdt` y `.dsk` (los que existan), de modo que el ejecutable queda a mano en la carpeta del ejemplo; esas copias están ignoradas por git y `make clean` las elimina. El framework nunca se copia al repositorio. `make run` abre el CDT del ejemplo, que autorrancia, o su DSK, con el ejecutable indicado en `GLASNOST_EMULATOR` o, en su defecto, con AmSpiriT-Lite si está junto al repositorio y, si no, con MacAPE. Sin emulador disponible imprime la ruta de la imagen. El emulador también puede fijarse en la propia orden: `make run GLASNOST_EMULATOR=/ruta/al/emulador`.

Para lotes o automatización sigue disponible la herramienta de bajo nivel:

```
python3 tools/build_examples.py minimal
python3 tools/build_examples.py animation vertical
```

Los resultados quedan en `.build/<nombre>/`. Según el manifiesto aparecen:

- `obj/<nombre>.bin`: binario AMSDOS.
- `<nombre>.cdt`: cinta, salvo ejemplos que declaren solo DSK.
- `selected-modules.json`: prueba de qué módulos entraron realmente.

`tools/build_examples.py` es la herramienta de bajo nivel y CPCTelera puede dejar un DSK intermedio por ejemplo. La política pública de `make examples` los sustituye por el único DSK común y deja su catálogo reproducible en `.build/tests/catalog.json`.

3.3 Probar manualmente en un CPC o emulador

1. Ejecute `make examples`.
2. Inserte `.build/tests/GlasnostFWK-tests.dsk` o el CDT específico en el CPC/emulador.
3. En DSK ejecute `CAT` y cargue el binario cuyo nombre coincide con el ejemplo; por ejemplo, `RUN"VERTICAL"`, `RUN"TILEREND"`, `RUN"SPRITECL"` o `RUN"PLATGAME"`. Los nombres de más de ocho caracteres se ajustan a AMSDOS: `resources128` es `RESOURCE.BIN`, `tilerender` es `TILEREND.BIN`, `spriteclip` es `SPRITECL.BIN`, `spritemask` es `SPRITEMA.BIN`. El ejemplo `platformer` declara el nombre `PLATGAME` en su manifiesto.
4. Para la demostración jugable `vertical`, mantenga CursorRight y recoja los tres puntos. `tilecoll`, `tilerender`, `spriteclip`, `spritemask` no necesitan entrada: muestran automáticamente su resultado.

> Los smoke automáticos no se limitan a comprobar que el emulador sigue abierto: leen RAM, memoria de vídeo, teclado y, en audio, registros PSG.

4. Estructura del repositorio

Ruta	Responsabilidad
`src/core`	Sistema, entrada y sistemas ECS (manager, behavior, movimiento, entrada, plataforma, colisión y animación).
`src/video`	Mode 0, CRTC, direcciones, limpieza y sprites con clipping, máscaras opcionales y orden explícito.
`src/ui`	Fuente sprite Mode 0, números pequeños y ventanas con fondo restaurable.
`src/audio`	Adaptador opcional Arkos Tracker 3 AKG.
`src/resources`	Bancos 128 KB, AMSDOS y ZX7B.
`src/extensions`	Behavior, física, tilemaps y render de tilemap opcionales.
`examples`	Consumidores independientes y sus fuentes de juego.
`profiles`, `manifests`	Geometría y recursos declarativos.
`generated`	Tablas producidas por herramientas; no son la fuente canónica.
`tools`, `tests`	Construcción, generación, paquetes y validación.

La dirección de dependencias es `juego → extensión opcional → núcleo → CPCTelera/hardware`. Ningún archivo de `src/` puede importar un ejemplo o símbolos de juego.

5. Crear un proyecto o ejemplo

5.1 Directorio mínimo

```
examples/mi_juego/
├── example.json
└── main.s
```

El manifiesto mínimo selecciona un perfil, módulos y fuentes propias:

```
{
  "name": "mi_juego",
  "profile": "standard-160x200.json",
  "transparency": {"pen": 0},
  "extended_memory": false,
  "modules": [
    "core/system.s",
    "core/input.s",
    "video/video.s"
  ],
  "game_sources": ["main.s"]
}
```

Después añada el nombre a `EXAMPLES` en el `Makefile` si debe formar parte de la construcción completa y del paquete oficial.

5.2 Campos del manifiesto

Campo	Uso
`name`	Nombre del proyecto y de sus artefactos.
`profile`	JSON de vídeo situado en `profiles/`.
`modules`	Fuentes del framework relativos a `src/`.
`game_sources`	Fuentes `.s` / `.inc` del consumidor. Debe incluir `main.s`.
`transparency`	Opcional: `{"pen": 0..15}`. Genera la tabla Mode 0 en `0x0100` y habilita las APIs transparentes. Los assets de texto incorporados requieren pen 0.
`extended_memory`	Opcional, booleano. Habilita bancos, reserva stub y scratch en `0x0200..0x0215` y mueve el código a la primera dirección libre, `0x0216`.
`cpct_modules`	Fuentes concretos de CPCTelera que deben compilarse, por ejemplo el player AKG.
`media`	Lista con `"dsk"`, `"cdt"` o ambos. Por defecto genera ambos.
`disk_files`	Payloads adicionales para DSK: fuente, codificación, nombre AMSDOS, símbolo y compresión.

Las rutas absolutas, recorridos `..`, ficheros inexistentes, nombres AMSDOS inválidos, símbolos duplicados y combinaciones de capacidades inválidas se rechazan antes de construir. Sin memoria extendida el código comienza en `0x0200`; con ella comienza en `0x0216`, justo después del scratch bancario. La detección usa temporalmente `0x4000`, restaura RAM normal y banco 0 y no reserva una dirección dentro de la ventana bancaria.

5.3 Configuración CPCtelera y assets

Un ejemplo puede incluir su propia configuración de proyecto CPCtelera en `cfg/build_config.mk`. Si existe, el constructor la usa como base en lugar de `cfg/build_config.template.mk`, de modo que los flags, herramientas y rutas que añada se conservan. Solo se reescriben los valores que gobierna el manifiesto: `CPCT_PATH`, `PROJNAME`, `TARGET`, `Z80CODELOC` y el `-I$(SRCDIR)` del ensamblador.

La carpeta `assets/` del ejemplo se copia a `src/assets` del proyecto generado. Sus `.s` se ensamblan sin declararlos en `game_sources`, los `.inc` se incluyen como `assets/<archivo>.inc` y los originales se copian para las herramientas. Además, si existe `assets/`

`assets.json`, el constructor ejecuta `tools/import_assets.py` antes de compilar: convierte PNG Mode 0 con `img2cpc`, lee capas CSV de Tiled con la misma reindexación que `cpct_tmx2csv` y puede comprimir mapas con `zx7b`.

La conversión es incremental como en CPCtelera. El pipeline guarda en `assets/generated/pipeline.json` un SHA-256 del manifiesto, la paleta y cada PNG/TMX, más el hash de las salidas. Si nada cambió, `make` no invoca ninguna herramienta; si se edita un original, regenera `assets.s` y `assets.inc`. `make assets` ejecuta solo esa comprobación:

```
cd examples/platformer
make assets     # convierte PNG/TMX solo si cambiaron
make            # convierte si hace falta y compila
```

`examples/platformer` es la referencia completa: PNG de tiles y sprites, `level.tmx` con dos capas, paleta Mode 0 definida en códigos de firmware y cargada al inicializar, mapa comprimido con `zx7b` y descompresión única al inicializar mediante `resources/zx7b.s`. La música no se enlaza por defecto para no alterar el presupuesto de frame ni el smoke; `assets/README.md` documenta cómo reutilizar la canción MIT de `examples/audio`. El resto de ejemplos siguen usando la plantilla compartida y no copian assets.

5.4 Fuente principal

```
.module mi_juego
.globl fwk_project_init, fwk_system_wait_frame
.globl fwk_video_init

.area _CODE
_main::
    call fwk_project_init
    call fwk_video_init
loop:
    call fwk_system_wait_frame
    jr loop
```

`fwk_project_init` inicializa el sistema e instala una sola vez las capacidades declaradas: tabla de transparencia y detección bancaria. Declare con `.globl` únicamente las funciones que use. Los módulos ausentes del manifiesto no se copian ni enlazan.

6. Referencia de módulos

6.0 ABI y entity manager ECS 2.0

La rama `ecs-v2` documenta el contrato binario aprobado en `docs/ecs-abi-2.0.md`. Define entidades AoS de slots estables, máscara de componentes de 16 bits, estados separados de lifecycle, offsets generados, managers múltiples y llamadas Z80 con carry y código en A. La fase 2 implementa el manager en `core/ecs_manager.s`.

La entidad empieza por máscara, estado, tipo, X, Y, anchura y altura; después incorpora los bloques de componentes y campos propios del juego. X y anchura se expresan en bytes de pantalla, Y y altura en scanlines. El tamaño máximo es 128 bytes para que todos los campos sean accesibles mediante desplazamientos indexados Z80. Render ocupa siempre el bit 0 del perfil, aunque una entidad concreta puede no activarlo.

La fase 3 genera el perfil desde el objeto `ecs` del manifiesto. `x_format` e `y_format` admiten `u8`, `u16` o `fixed_8_8`. Los componentes propios declaran dependencias y campos; los tipos disponibles son `u8`, `i8`, `u16`, `i16`, `fixed_8_8`, `pointer` y `bytes[n]`. `velocity_x` y `velocity_y` resuelven automáticamente una velocidad compatible con la coordenada.

```
"ecs": {
  "x_format": "u8",
  "y_format": "u8",
  "components": ["behavior", "actor"],
  "custom_components": [
    {
      "name": "actor",
      "dependencies": ["behavior"],
      "fields": []
    }
  ],
  "fields": [{"name": "health", "type": "u8"}]
}
```

`RENDER` se inserta siempre como bit 0. El resto conserva el orden solicitado, insertando antes sus dependencias. Se rechazan nombres o tipos inválidos, duplicados, ciclos, componentes desconocidos, más de 16 componentes, coordenadas incapaces de representar el perfil de vídeo y entidades mayores de 128 bytes.

> **Portfolio actual.** `RENDER`, `MOTION`, `BEHAVIOR`, `INPUT` y `PLATFORM`, `COLLISION` y `ANIMATION` tienen layout normativo, y el renderer ECS dirty/multi-manager está en `video/ecs_render.s`.

El include versionado está en `generated/ecs_profile.inc`. Antes de cambiarlo puede obtener una vista JSON de componentes resueltos, dependencias insertadas, campos, offsets, tamaños y diferencias sin escribir:

```
python3 tools/gen_ecs_profile.py examples/ecsmanager/example.json \
  --output examples/ecsmanager/generated/ecs_profile.inc --preview
make generate-ecs   # escribe el include después de revisar la vista previa
make check CHECK_EXAMPLES=ecsmanager  # falla si el include vuelve a quedar
obsoleto
```

El generador publica `ECS_ABI_MAJOR/MINOR`, formatos y tamaños X/Y, `ECS_C_*`, `ECS_COMPONENT_VALID_MASK`, todos los offsets `ECS_E_*` y `ECS_ENTITY_SIZE`. Incluya `core/ecs_manager.s` en el manifiesto y `core/ecs_manager.inc` en el ensamblador. Un descriptor reserva ocho bytes y referencia almacenamiento propiedad del juego:

```
.include "ecs_profile.inc"
.include "core/ecs_manager.inc"

entities: .ds ECS_ENTITY_SIZE * 16
FWK_ECS_MANAGER_DESCRIPTOR game_manager, entities, 16
```

Sistema ECS de behavior — `core/ecs_behavior.s`

Requiere los componentes `behavior` y `motion`, coordenadas `u8` y el módulo `extensions/behavior.s`. `fwk_ecs_system_behavior` recibe `IX=manager`, ejecuta una vez cada entidad activa con `BEHAVIOR` y copia de vuelta las velocidades, el puntero de programa y el temporizador. El programa puede ser compartido; el adaptador usa un registro temporal privado y no es reentrante.

Sistema ECS de movimiento — `core/ecs_motion.s`

Requiere el componente `motion` y coordenadas `u8`. `fwk_ecs_system_motion_u8` recibe `IX=manager`, suma las velocidades con signo a X/Y, satura contra `ECS_X_MAX/ECS_Y_MAX` y activa `ECS_RENDER_DIRTY` cuando cambia la posición. El desbordamiento se detecta con acarreo, de modo que coordenadas por encima de 127 son válidas. Las entidades con el componente `platform` se omiten: su movimiento y colisión pertenecen a `fwk_ecs_system_platform`. No implementa gravedad, colisión ni física 8.8.

Colisión ECS — `core/ecs_collision.s`

Requiere el componente `collision` (un byte `ECS_E_COLLISION_CONTACT` por entidad) y coordenadas `u8` (el build falla con otro formato). `fwk_ecs_collide` comprueba **un sujeto contra los objetivos de un manager**: `IX` =manager, `IY` =sujeto, `BC` =máscara de componentes que deben tener los objetivos, `HL` =callback, `E` =canal. Llama al callback por cada objetivo `ACTIVE` cuya caja AABB solape con la del sujeto (solape estricto, `u8`, posiciones dentro de pantalla, anchura y altura distintas de cero y sumas hasta 255). El callback recibe `IX` =manager, `IY` =objetivo, `HL` =sujeto y `A` = `ENTER`, `STAY` o `EXIT`; con ese `IX`/`IY` puede llamar directamente a `fwk_ecs_entity_mark_destroy`.

El canal es un bit de `ECS_E_COLLISION_CONTACT` que guarda si el sujeto tocaba algo en la pasada anterior de ese canal: `ENTER` si no tocaba, `STAY` si ya tocaba y, al dejar de tocar, una única llamada `EXIT` con `IY=0`. **Cada grupo de objetivos que pruebes contra el mismo sujeto necesita su propio canal** (ocho disponibles, `E` con exactamente un bit); con uno compartido, una pasada sin contactos borra el estado de la otra y provoca `EXIT` y `ENTER` falsos en cada frame. `E=0` no guarda estado y todo contacto llega como `ENTER`. El campo `ECS_E_COLLISION_CONTACT` de las plantillas debe ser 0.

Efecto por objetivo o por contacto. El estado es por sujeto y canal, no por objetivo: una moneda nueva que empieza a tocarse mientras el canal ya estaba activo llega como `STAY`. Por eso recoger una moneda actúa en toda llamada salvo `EXIT`, mientras que perder una vida (una vez por contacto) actúa en `ENTER`; un segundo enemigo tocado sin haber dejado de tocar al primero no cuesta otra vida Un carry activo del callback detiene la pasada y no actualiza el estado.

No hay tabla de pares, buffer ni límite de entidades: unos 210 bytes de código y 7 de datos (presupuesto 300 y 8, vigilado por `tests/test_ecs_collision.py`). Límites: no es reentrante (ni siquiera una segunda llamada que devuelva un error: sustituye el callback, el sujeto y el canal al entrar), no comprueba el estado del sujeto (si un callback lo destruye, sigue probándose contra el resto de la pasada) y un objetivo destruido en `ENTER` produce un `EXIT` sin objetivo en la pasada siguiente, que el callback debe tolerar. Detalle normativo en `docs/ecs-abi-2.0.md`, sección 9.

```
; el jugador contra monedas (canal 1) y contra enemigos (canal 2)
    ld iy, (jugador)
    ld ix, #monedas
    ld bc, #ECS_C_COIN
    ld e, #0x01
    ld hl, #al_tocar_moneda
    call fwk_ecs_collide
    ld ix, #enemigos
    ld bc, #ECS_C_ENEMY
    ld e, #0x02
    ld hl, #al_tocar_enemigo
    call fwk_ecs_collide
    ; ...

al_tocar_moneda:                  ; IX=manager IY=moneda HL=jugador
A=tipo
    cp #ECS_COLLISION_EXIT
    jr nz, recoger
    xor a                         ; EXIT (IY=0): nada que hacer
    ret
recoger:
    jp fwk_ecs_entity_mark_destroy ; ENTER o STAY: la moneda
desaparece, carry limpio

al_tocar_enemigo:                 ; una vida menos solo al empezar el
contacto
    cp #ECS_COLLISION_ENTER
    jp z, quitar_vida             ; el juego: resta la vida y termina
con xor a / ret
    xor a
    ret
```

Animación ECS — `core/ecs_animation.s`

Requiere el componente `animation` (tres campos por entidad: `ECS_E_ANIMATION_DESCRIPTOR`, `FRAME` y `TIMER`). El descriptor es de solo lectura, compartible y tiene este formato (`core/ecs_animation.inc`): byte `count` (1 a 255), byte `duration` (1 a 255) y `count` punteros de sprite. `fwk_ecs_system_animation` recibe `IX`=manager y recorre las entidades `ACTIVE` con el componente y un descriptor: cada llamada decrementa su `TIMER`; al llegar a cero lo recarga con la duración, avanza el frame (vuelve a 0 tras el último), pone `ECS_E_RENDER_SPRITE` al puntero de ese frame y activa `ECS_RENDER_DIRTY`. Una entidad sin descriptor no se toca.

`fwk_ecs_anim_set` (`IY`=entidad, `HL`=descriptor o 0) cambia de animación entre recorridos: un descriptor distinto reinicia en el frame 0 con el temporizador igual a la duración, pone el sprite y activa `DIRTY`; `HL=0` detiene la animación y conserva el sprite actual; elegir el descriptor ya activo no cambia nada mientras el temporizador no sea cero, de modo que puede llamarse cada frame con el estado actual del jugador (andar, quieto). Con el temporizador a cero (una plantilla que ya trae el descriptor) arranca. Inicia siempre las entidades nuevas con `fwk_ecs_anim_set` tras crearlas.

Límites. `count` y `duration` van de 1 a 255; con 0 el comportamiento es indefinido y silencioso. Si llamas a `fwk_ecs_anim_set` antes del sistema en el mismo frame, el frame 0 dura una llamada menos (con duración 1 no llega a verse): llámalo después para la duración completa. El descriptor no lleva el tamaño del sprite, así que `ECS_E_WIDTH` y `ECS_E_HEIGHT` no cambian; si cambias el sprite por tu cuenta y el descriptor ya estaba activo, no se restituye hasta el siguiente avance. Una animación de un solo frame vuelve a fijar el sprite y a activar `DIRTY` cada `duration` llamadas: para una pose fija usa duración 255 o detén la animación con `HL=0` tras elegirla. `fwk_ecs_anim_set` no comprueba el estado de la entidad. Ocupa 109 bytes de código y ningún dato (presupuesto 150 y 0, vigilado por `tests/test_ecs_animation.py`). Detalle normativo en `docs/ecs-abi-2.0.md`, sección 10.

```
; una vez por frame: primero el sistema, después elegir la animación
del
; estado actual (así cada frame nuevo dura la duración completa)
    ld ix, #entidades              ; IX = manager con las entidades
animadas
    call fwk_ecs_system_animation
    ld iy, (jugador)
    ld hl, #anim_andar             ; descriptor: .db 4, 3 / .dw
frame0..frame3
    ld a, (jugador_quieto)
    or a
    jr z, elegida
    ld hl, #anim_quieto
elegida:
    call fwk_ecs_anim_set          ; no reinicia si ya estaba en marcha
```

Renderer ECS — `video/ecs_render.s`

Restaura y redibuja solo lo que cambió, con entidades de uno o varios managers, en un único orden global por Y. Requiere coordenadas `u8` (el build falla con otro formato), el módulo `video/sprite.s` y, por entidad con el bit `RENDER`, un buffer de fondo propio de al menos anchura por altura bytes (campos `ECS_E_RENDER_BACKGROUND` y `BACKGROUND_CAP`); el renderer no lo comprueba ni busca duplicados.

Una escena (`FWK_ECS_RENDER_SCENE nombre, tabla, cuenta, trabajo, capacidad`) agrupa una tabla de punteros a managers y un array de trabajo de `capacidad` entradas de 8 bytes que reserva el proyecto (una por entidad con `RENDER` visible o con fondo capturado). Cada frame se llama, con `IX`=escena y en este orden: `fwk_ecs_render_prepare` antes del VSYNC (reúne candidatas, marca las afectadas, bloquea los managers), `fwk_ecs_render_restore` tras el VSYNC (repone los fondos en orden inverso al último dibujo) y `fwk_ecs_render_draw` justo después (captura el fondo, dibuja en orden estable por `Y + SORT_OFFSET_Y`, actualiza el estado anterior, limpia `DIRTY` y libera los bloqueos). Una entidad se redibuja si tiene `DIRTY`, si debe aparecer o desaparecer, o si su caja (rectángulos anterior y actual) solapa con la de otra afectada, de forma transitiva. Los sistemas que cambian posición, sprite o visibilidad activan `DIRTY`; `fwk_ecs_entity_enable` también.

Un frame fuera de orden devuelve `FWK_ECS_ERR_RENDER`; un manager bloqueado, `FWK_ECS_ERR_BUSY`; un array de trabajo pequeño, `FWK_ECS_ERR_OVERFLOW`. Posiciones dentro de pantalla con sumas de posición y tamaño hasta 255 (el recorte por la derecha y abajo lo hace el módulo de sprites). Los empates de clave se resuelven por orden de tabla y slot. Un error en `restore` o `draw` es fatal (la escena queda a medias y bloqueada; no hay aborto), y cada manager debe aparecer una sola vez en la tabla. Ocupa 1.139 bytes de código (1.150 con transparencia activa) y 31 de datos (presupuesto 1.150 y 40, algo más de la mitad del renderer anterior, vigilado por `tests/test_ecs_render.py`). Detalle normativo en `docs/ecs-abi-2.0.md`, sección 8.

```
; declaraciones (el array de trabajo y un buffer de fondo por entidad
los pones tú)
managers:  .dw entidades
trabajo:   .ds 8*8                ; 8 entradas de 8 bytes
FWK_ECS_RENDER_SCENE escena, managers, 1, trabajo, 8
; en la inicialización: dibujar el fondo una vez; después, cada frame
    ld ix, #entidades
    call fwk_ecs_system_motion_u8   ; los sistemas activan DIRTY al
cambiar
    ld ix, #escena
    call fwk_ecs_render_prepare     ; antes del VSYNC
    call fwk_system_wait_frame
    ld ix, #escena
    call fwk_ecs_render_restore     ; solo VRAM, tras el VSYNC
    ld ix, #escena
    call fwk_ecs_render_draw
```

Sistema ECS de entrada — `core/ecs_input.s`

`fwk_ecs_system_input` recibe `IX=manager`, recorre entidades con `INPUT` y traduce el estado global de `fwk_input_update` a velocidad horizontal y salto. El campo `action_mask` usa izquierda `0x01`, derecha `0x02` y salto `0x04`. El sistema conserva el contrato de callback del manager y marca render dirty al iniciar un salto.

Sistema ECS de plataforma — `core/ecs_platform.s`

`fwk_ecs_system_platform` adapta cada entidad `PLATFORM` al estado legacy de siete bytes, llama a `fwk_physics_platform_step` y a `fwk_platform_tilemap_resolve`, y copia X/Y, velocidades y flags de vuelta. La entidad aporta punteros a la configuración física y al descriptor de tilemap. No resuelve colisión entre cuerpos.

Operaciones del manager

- `fwk_ecs_manager_init`: `IX=manager`; valida storage, capacidad y stride, marca todos los slots `FREE` e inicializa contadores y locks.
- `fwk_ecs_entity_create`: `IX=manager`, `HL=plantilla ACTIVE`; copia el registro completo al primer slot libre y devuelve `IY=slot`.
- `fwk_ecs_entity_disable`, `fwk_ecs_entity_enable` y `fwk_ecs_entity_mark_destroy`: aplican únicamente las transiciones válidas y mantienen `ACTIVE_COUNT`.
- `fwk_ecs_entity_set_mask`: `BC=máscara`; rechaza bits ajenos al perfil y no permite retirar render mientras exista un fondo capturado.
- `fwk_ecs_system_run`: `BC=máscara requerida`, `HL=callback`; recorre slots activos en orden. El callback recibe `IX=manager` e `IY=entidad`, debe preservarlos y puede marcar destrucción.
- `fwk_ecs_manager_collect`: convierte `PENDING_DESTROY` en `FREE` y devuelve en `B` cuántos slots liberó.

Éxito significa carry limpio y `A=0`; error significa carry activo y un código `FWK_ECS_ERR_*` en A. Las operaciones estructurales se rechazan durante locks. El recorrido usa un `CALL` automodificable, por lo que no es reentrante, no puede anidarse ni ejecutarse desde IRQ. La API valida descriptores, máscaras y pertenencia de punteros también en esta primera implementación; no existe coste de `generation` en cada acceso.

6.1 Sistema — `core/system.s`

- `fwk_system_init`: desactiva firmware, instala IRQ y deja el callback nulo. Altera `AF,BC,DE,HL`.
- `fwk_system_wait_frame`: espera VSYNC. Altera `AF,BC,DE,HL`.
- `fwk_system_set_irq_callback`: recibe `HL=callback|0`. El callback se ejecuta una vez por cada seis IRQ, después del escaneo del teclado.
- Estado público: `fwk_system_irq_count`, `fwk_system_frame_count` y el puntero de callback.

No cambie el callback mientras está activo sin deshabilitar interrupciones. Un callback debe preservar lo que necesite el programa principal.

6.2 Entrada — `core/input.s`

Admite hasta ocho acciones lógicas y separa pulsación sostenida, flanco de entrada y flanco de salida.

```
bindings:
    .dw Key_CursorRight
    .db 0x01
    .dw Key_CursorLeft
    .db 0x02
    .dw 0
    .db 0
```

- `fwk_input_init`: borra el estado.
- `fwk_input_set_bindings`: `HL=tabla`. La tabla debe persistir y termina con máscara cero.
- `fwk_input_update`: actualiza `fwk_input_held`, `fwk_input_pressed` y `fwk_input_released`. Altera `AF,BC,DE,HL,IY`.

6.3 Vídeo Mode 0 — `video/video.s`

- `fwk_video_init`: selecciona Mode 0.
- `fwk_video_configure_crtc`: aplica la tabla CRTC del perfil. Usarlo no convierte un perfil experimental en verificado.
- `fwk_video_get_ptr`: `HL=y` de 16 bits y `C=x` en bytes; devuelve `HL=puntero`.
- `fwk_video_plot_byte`: añade `A=byte empaquetado Mode 0`.
- `fwk_video_clear_visible`: `A=patrón`; limpia solo regiones visibles y conserva huecos.

`fwk_video_width_bytes` y `fwk_video_height` exponen la geometría activa para módulos de validación. El ancho estándar es 80 bytes, equivalentes a 160 píxeles en Mode 0. El llamador debe mantener las coordenadas dentro del perfil.

6.4 Sprites Mode 0 — `video/sprite.s`

```
FWK_SPRITE_DESCRIPTOR player_sprite, player_pixels, 20, 40, 6, 8
player_background: .ds 6 * 8
```

El descriptor de ocho bytes contiene puntero a datos, X/Y con signo de 16 bits, ancho en bytes Mode 0 y alto en scanlines. Las coordenadas negativas permiten entrar o salir por arriba e izquierda; los datos son `width*height` bytes por filas y sin padding.

- `fwk_sprite_draw`: `IX=descriptor`; valida, recorta contra los cuatro bordes y dibuja el rectángulo visible de forma opaca.
- `fwk_sprite_draw_recolor`: `IX=descriptor`, `D=pen origen`, `E=pen sustituto`; reemplaza un pen Mode 0 sin transparencia.
- `fwk_sprite_draw_masked`: `IX=descriptor`; conserva los píxeles cuyo pen coincide con `transparency.pen` y copia los demás mediante la tabla implícita de `0x0100`. El símbolo solo existe cuando el proyecto activa transparencia.
- `fwk_sprite_draw_masked_recolor`: `IX=descriptor`, `D=pen origen`, `E=pen sustituto`; sustituye un pen Mode 0 durante el dibujo y aplica después la transparencia configurada. Rechaza valores fuera de 0..15 antes de escribir.
- `fwk_sprite_capture`: `IX=descriptor`, `DE=buffer`; guarda el fondo visible y devuelve `IY` al final.
- `fwk_sprite_restore`: `IX=mismo descriptor`, `HL=buffer`; restaura el fondo visible y devuelve `IY` al final.

Carry indica pens o dimensiones inválidos y span de datos con wrap; una imagen totalmente exterior es válida y no escribe. Reserve `width*height` bytes para los datos. Los dibujos preservan `IX` e `IY` y alteran `AF,BC,DE,HL`; captura y restauración preservan `IX` y alteran además `IY`. Ninguna es reentrante. `fwk_sprite_draw_recolor` ofrece la misma sustitución sin máscara. El recoloreado sustituye un solo pen, no remapea toda la paleta; al ejecutarse antes de la composición, sustituir por el color clave vuelve transparentes esos píxeles. Véase `docs/sprites.md`.

6.5 Texto — `ui/text.s`

El módulo usa la fuente gráfica de los juegos originales: cada carácter es un sprite Mode 0 enmascarado de 2 bytes por 9 líneas. Incluye además los dígitos pequeños de 2 bytes por 5 líneas. No depende del dibujado de caracteres de CPCTelera.

```
call fwk_project_init       ; instala tabla pen 0 en &0100
call fwk_video_init
call fwk_text_init          ; selecciona las fuentes incorporadas
xor a                       ; color 0 de la tabla original
call fwk_text_set_color

ld hl, #40                  ; y
ld c, #10                   ; x en bytes Mode 0
call fwk_video_get_ptr
ld iy, #message
call fwk_text_draw

ld hl, #56
ld c, #10
call fwk_video_get_ptr
ld a, #123
call fwk_text_draw_small_u8
```

- `fwk_text_init`: selecciona los assets incorporados. El manifiesto debe declarar `transparency.pen=0`; `fwk_project_init` copia la tabla una sola vez antes de esta llamada.
- `fwk_text_set_color`: `A=0..14`; carry indica un índice inválido. Los índices reproducen esta secuencia de los juegos: blanco, magenta, naranja, azul brillante, rojo brillante, verde brillante, amarillo, verde, amarillo brillante, cian brillante, azul cielo, negro, blanco, rojo y rosa. El color visible depende de la paleta que el juego haya asignado a sus pens; pen 14 queda reservado como sombra fija.
- `fwk_text_set_colors`: alias de migración que conserva el símbolo y el registro de entrada; usa `L=0..14` e ignora `H`. Los antiguos números de pen no conservan su significado.
- `fwk_text_set_assets`: `HL=fuente normal`, `DE=dígitos pequeños`. Espera 51 frames consecutivos de 18 bytes y 10 frames consecutivos de 10 bytes.
- `fwk_text_draw`: `IY=cadena NUL`, `HL=puntero de vídeo`. Admite espacio, `!`, `-`, `/`, `0..9`, `:`, `>`, `?`, `@` y `A..Z`.
- `fwk_text_draw_small_digit`: `A=0..9`, `HL=vídeo`; carry indica dígito inválido.
- `fwk_text_draw_small_u8`: `A=0..255`, `HL=vídeo`; imprime decimal sin ceros iniciales.

Los caracteres no admitidos dejan una celda en blanco. Todas las rutinas dibujan sin clipping: el consumidor debe mantener cada sprite dentro de la pantalla. El renderer recolorea en un

buffer interno de 18 bytes y no es reentrante. Como el enmascarado conserva el fondo, para sustituir texto ya dibujado hay que restaurar o limpiar primero su rectángulo.

6.6 Ventanas restaurables — `ui/window.s`

```
message_window:
    .db 10       ; x en bytes Mode 0
    .dw 40       ; y de 16 bits
    .db 20, 16   ; ancho y alto
    .db 0xFF     ; patrón de borde
    .db 0x00     ; patrón de relleno

window_buffer: .ds 20 * 16
```

- `fwk_window_capture`: `IX=descriptor`, `DE=buffer`; guarda `width*height` bytes.
- `fwk_window_draw`: `IX=descriptor`; dibuja borde de un byte y relleno.
- `fwk_window_restore`: `IX=descriptor`, `HL=buffer`; repone exactamente la captura.

Carry indica geometría inválida. El módulo usa las dimensiones y tabla de líneas del perfil; no contiene texto, entrada, espera ni un buffer global. El llamador debe reservar suficiente memoria y conservar la geometría entre captura y restauración. El contrato detallado está en `docs/windows.md`.

6.7 Comportamiento — `extensions/behavior.s`

Intérprete de programas enlazados para comportamientos. `fwk_behavior_run` recibe en `IX` un registro de 8 bytes: `x,y,vx,vy,flags`, puntero de programa de 16 bits y temporizador. El intérprete cambia estado y velocidades, no la posición. En un juego ECS no se llama directamente: `fwk_ecs_system_behavior` (`core/ecs_behavior.s`) copia los campos de cada entidad con `BEHAVIOR` a un registro privado, ejecuta el intérprete y los devuelve; después `fwk_ecs_system_motion_u8` aplica las velocidades y activa `DIRTY`.

```
enemy_program:
    FWK_BEHAVIOR_SET_VY 2
    FWK_BEHAVIOR_WAIT 49, enemy_turn
enemy_turn:
    FWK_BEHAVIOR_SET_VY -2
    FWK_BEHAVIOR_WAIT 49, enemy_program

    ; cada turno, con IX=manager que contiene al enemigo
    push ix
    call fwk_ecs_system_behavior
    pop ix
    call fwk_ecs_system_motion_u8
```

`FWK_BEHAVIOR_WAIT` fija el temporizador, ejecuta una espera no bloqueante y termina internamente su tabla de condiciones. Los ticks válidos son 1–255; cero desborda a 255 en el primer decremento. Las acciones instantáneas pueden encadenarse en el mismo frame, con un máximo de 16 despachos. Las tablas escritas a mano deben acabar con `FWK_BEHAVIOR_CONDITIONS_END`.

Acciones: `idle`, `wait`, `set_timer`, `set_vx`, `set_vy` y `drive_vx`. Condiciones incorporadas: `true` y `timeout`. Una condición personalizada recibe `IX=registro`, debe preservar `IX/DE` y devuelve verdadero con Z activo. Puntero de programa cero desactiva el comportamiento. El contador de despachos es global: no llame al intérprete de forma anidada ni desde una interrupción.

Los tres enemigos de `platformer` son el ejemplo ejecutable: cada entidad enemiga lleva los componentes `MOTION`, `BEHAVIOR`, `COLLISION` y `ANIMATION`. El horizontal usa tramos temporizados para recorrer X=32..54; los dos verticales reutilizan el mismo programa inmutable de 49 ticks, pero cada entidad conserva su propio cursor y temporizador. El horizontal y el vertical derecho viven en managers propios que el bucle de juego procesa en frames alternos, mientras el vertical izquierdo comparte manager con el jugador y se actualiza en cada frame.

6.8 Física de plataforma — `extensions/physics_platform.s`

`fwk_physics_platform_step` recibe `IY=estado` e `IX=configuración`. El estado de siete bytes contiene `x,y,vx,vy,width,height,flags`; la configuración define ancho del mundo, coordenada inclusiva del suelo, gravedad y caída máxima.

Aplica velocidad byte con signo, límites horizontales, gravedad, techo, suelo y los bits `FWK_PLATFORM_AIRBORNE` y `FWK_PLATFORM_TILE_GROUNDED`. No conoce mapas, tiles ni sprites. Altera `AF,BC`, preserva `IX/IY` y es reentrante si el estado no se comparte. El consumidor debe usar dimensiones no nulas, `width <= world_width` y parámetros que no desborden la velocidad con signo.

6.9 Consulta de tilemap — `extensions/tilemap.s`

```
tile_types: .db FWK_TILE_PASSABLE, FWK_TILE_SOLID, FWK_TILE_ONE_WAY
tile_data:  .db 0, 1, 0, 2
FWK_TILEMAP_DESCRIPTOR level_map, tile_data, tile_types, 4, 1, 2, 3, 4,
8, 3
```

`fwk_tilemap_query` recibe `IX=descriptor`, `B=Y` y `C=X`. Devuelve el tipo de colisión en `A` con carry limpio; fuera de mapa, descriptor inválido o id fuera de tabla devuelve `A=0` y carry activo. Preserva `BC,IX,IY`, altera `AF,DE,HL` y no escribe en los datos.

El mapa es row-major y las coordenadas son de 8 bits. Los tamaños de tile deben ser potencias de dos coherentes con sus shifts; el módulo valida valores nulos y shifts fuera de rango, pero no esa coherencia. No incluye render, edición, streaming, habitaciones ni portales.

6.10 Colisión de plataforma con tilemap — `extensions/platform_tilemap.s`

Llame a `fwk_platform_tilemap_resolve` inmediatamente después de `fwk_physics_platform_step`, con `IY=estado` e `IX=tilemap`. Corrige posición, velocidad y flags. Devuelve en `A` una máscara de izquierda, derecha, techo, suelo y pérdida de apoyo.

Los sólidos bloquean todos los ejes. `FWK_TILE_ONE_WAY` deja pasar al subir y solo bloquea un cruce descendente o sostiene un cuerpo ya apoyado. Las consultas inválidas pasan. No hay barrido continuo: la velocidad no debe atravesar un tile completo. Solo consulta las esquinas del rectángulo y no resuelve pendientes ni colisión entre cuerpos. Véase `docs/tilemap-collision.md`.

6.11 Render de tilemap — `extensions/tilemap_render.s`

```
FWK_TILE_RENDER_DESCRIPTOR level_view, tileset, 0, 0, 10, 24, 12, 10,
16
; tileset, cámara X/Y, pantalla X/Y, viewport ancho/alto, gráficos
```

- `fwk_tilemap_render_view`: `IX=tilemap`, `IY=render`; prevalida cámara, pantalla, tileset e ids y dibuja todo el viewport.
- `fwk_tilemap_render_cell`: además `B=fila`, `C=columna` relativas al viewport; redibuja solo esa celda.

Cada gráfico contiene `x_size*y_size` bytes Mode 0 contiguos. El render opaco admite exclusivamente las geometrías nativas de CPCTelera Next: 2x4, 2x8, 4x4 y 4x8 bytes (4x4, 4x8, 8x4 y 8x8 píxeles en Mode 0), exige que Y esté alineada al alto y despacha a `cpct_drawTileAligned*_f_asm`. La variante transparente usa un único tileset con el pen clave del proyecto y la tabla implícita de `0x0100`.

Carry indica error antes de escribir. Ambas rutinas preservan `IX/IY`, alteran `AF,BC,DE,HL` y no son reentrantes. No hay clipping, scroll sub-tile, bancos ni dirty rectangles automáticos; el mapa no se modifica. Véase `docs/tilemap-render.md`.

6.12 Física 8.8 — `extensions/physics_8_8.s`

`fwk_physics_8_8_step` recibe `IY=estado` e `IX=descriptor`. Posición y velocidad son palabras 8.8 little-endian; el noveno byte conserva el resto sub-LSB de fricción. El descriptor aporta límites inclusivos y fricción de `0..255`.

La fricción sigue el vector para no curvar diagonales al detener antes el eje menor. Después integra y rebota por eje. Altera `AF,BC,DE,HL`, preserva `IX/IY` y no es reentrante porque usa cuatro palabras internas. No resuelve colisiones entre cuerpos ni evita por sí sola atravesar geometría con velocidades excesivas. Véase `docs/physics-profiles.md`.

6.13 Audio AKG — `audio/akg.s`

El manifiesto también debe incluir `audio/arkosTracker_PlayerAkg.s` en `cpct_modules`, y el juego debe aportar su canción y tabla de efectos exportadas por Arkos Tracker.

1. `fwk_audio_init_music`: `HL=canción`, `A=subsong`.
2. `fwk_audio_init_sfx`: `HL=tabla de efectos`.
3. `fwk_audio_start`: instala el tick AKG en el callback del sistema.
4. `fwk_audio_play_sfx`: `A=id`, `B=volumen invertido`, `C=canal 0..2`.
5. `fwk_audio_stop`: para el player y retira el callback.

El player y sus datos deben permanecer visibles durante la IRQ. El coste total variable del player no está medido; no se promete 50 Hz para cualquier canción o escena.

6.14 Bancos de 128 KB — `resources/bank.s`

Active `"extended_memory": true`. El constructor añade este módulo, reserva `0x0200..0x0215`, sitúa el código desde `0x0216` y `fwk_project_init` ejecuta la detección una sola vez.

- `fwk_bank_init`: detecta RAM adicional. Devuelve `A=1/NZ` o `A=0/Z` y actualiza `fwk_bank_has_128k`.
- `fwk_bank_copy_from`: `A=banco 0..3`, `HL=origen bancado`, `DE=destino normal`, `BC=tamaño`.
- `fwk_bank_copy_to`: mismo contrato, con origen normal y destino bancado.

El lado bancado debe estar entre `0x4000` y `0x7FFF`. El stub comienza en `0x0200`. La detección deshabilita interrupciones, usa temporalmente `0x4000`, restaura los dos bytes afectados y vuelve con interrupciones habilitadas. Las copias también restauran el mapeo normal antes de volver. Carry indica banco inválido.

6.15 Carga AMSDOS — `resources/disk.s`

- `fwk_disk_init`: reconecta AMSDOS. Debe llamarse antes de `fwk_project_init`.
- `fwk_disk_load`: `HL=nombre`, `B=longitud`, `DE=destino`. Carry y `BC=longitud lógica` indican apertura correcta.

Usa `0xF800..0xFFFF` como buffer de cabecera y no es reentrante. El llamador debe verificar longitud y contenido; este backend no ofrece por sí solo integridad.

6.16 Descompresión ZX7B — `resources/zx7b.s`

`fwk_zx7b_unpack` recibe `HL=último byte comprimido` y `DE=último byte del destino`. Escribe hacia atrás mediante `cpct_zx7b_decrunch_s_asm` y altera `AF,BC,DE,HL`.

No comprueba límites ni corrupción. Para archivos declarados en `disk_files`, el constructor genera constantes `FWK_DISK_*_SIZE` y `FWK_DISK_*_UNPACKED_SIZE`.

7. Perfiles y recursos

7.1 Perfil estándar 160×200

`profiles/standard-160x200.json` define Mode 0, 80 bytes por línea, 200 líneas, pantalla en `0xC000` y pila reservada en `0xB800..0xBFFF`. Es el perfil usado por los ejemplos.

7.2 Perfil experimental 160×288

`profiles/experimental-160x288.json` usa coordenadas verticales de 16 bits y dos segmentos de vídeo. Se han comprobado 23.040 direcciones únicas y el cruce de memoria; la temporización CRTC no se ha validado visualmente en varios CRTC ni en hardware.

7.3 Generar tablas

```
python3 tools/gen_video.py profiles/standard-160x200.json --out-dir
generated
python3 tools/gen_resources.py manifests/resources128.json --output
generated/resources128.inc
```

Edite los JSON fuente y regenere. No edite las tablas de `generated/` a mano. El generador de recursos rechaza solapamientos, destinos reservados, bancos no disponibles y alineaciones incorrectas.

7.4 Archivos adicionales en DSK

```
"disk_files": [{
  "source": "mapa.bin",
  "encoding": "raw",
  "name": "MAPA.BIN",
  "symbol": "mapa",
  "compression": "zx7b"
}]
```

`name` debe cumplir AMSDOS 8.3 en mayúsculas. `encoding` admite `raw` y `hex`; `compression`, `none` y `zx7b`. Los payloads se incorporan al DSK, no al ejecutable.

8. Ejemplos incluidos

Ejemplo	Qué demuestra	Medios
`minimal`	Sistema, entrada y vídeo mínimos.	BIN, DSK común, CDT
`board`	Entrada, texto y composición sencilla de tablero.	BIN, DSK común, CDT
`gravity`	Traza amarilla sobre negro: gravedad, salto, límites y aterrizaje con velocidad byte.	BIN, DSK común, CDT
`tilecoll`	Escenario cian y traza amarilla: consulta row-major, techo, pared y suelo sólido.	BIN, DSK común, CDT
`tilerender`	Viewport Mode 0: cambio de cámara, render completo y actualización de una celda.	BIN, DSK común, CDT
`spriteclip`	Sprites opacos: clipping en cuatro bordes y captura/restauración del fondo visible.	BIN, DSK común, CDT
`spritemask`	Color clave configurable: pen 3, tabla generada, composición, recoloreado 5→15, clipping y rechazo de pens/span.	BIN, DSK común, CDT
`physics88`	Traza cian sobre negro: integración 8.8, fricción vectorial y rebote.	BIN, DSK común, CDT
`video`	Direcciones y limpieza de regiones visibles.	BIN, DSK común, CDT
`resources128`	Detección y copia bancaria, con fallback de 64 KB.	BIN, DSK común, CDT
`diskload`	Carga AMSDOS cruda y ZX7B con comparación byte a byte.	BIN, DSK común

Ejemplo	Qué demuestra	Medios
`audio`	Música AKG, stop/restart y SFX.	BIN, DSK común, CDT
`window`	Validación, captura, caja, texto, restauración y reapertura.	BIN, DSK común, CDT
`vertical`	Escena jugable en ECS: entrada, movimiento, colisión, render y victoria.	BIN, DSK común, CDT
`perf`	Arnés de rendimiento: vacío, limpieza, sprites opacos/enmascarados, captura/restauración, lote de 8, tilemap completo y celda.	BIN, DSK común
`tilemask`	Render de tilemap con composición enmascarada por tile.	BIN, DSK común, CDT
`experimental`	Perfil experimental 160x288: configuración CRTC y limpieza del área extendida.	BIN, DSK común, CDT
`platformer`	Plataformas en ECS: jugador, cuatro monedas, tres enemigos, puerta, vidas, HUD y parpadeo al perder una vida, cuatro managers, el renderer selectivo y dos niveles.	BIN, DSK común, CDT
`ecsmanager`	Prueba técnica del manager ECS: slots estables, plantillas, máscaras, lifecycle, locks, recorrido y recolección.	BIN, DSK común, CDT
`ecscollision`	Prueba técnica de colisión ECS: reproduce un escenario generado por un modelo Python (ENTER, STAY, EXIT, máscara, destrucción, aborto, BUSY, callback nulo) y registra los argumentos de cada callback.	BIN, DSK común, CDT
`ecsanimation`	Prueba técnica de animación ECS: reproduce un escenario generado por un modelo Python (cuenta atrás, avance y vuelta de frame, cambio de descriptor,	

Ejemplo	Qué demuestra	Medios
	descriptor ya activo, parada con 0) y registra frame, temporizador, sprite y DIRTY de cada entidad.	BIN, DSK común, CDT
`ecsrender`	Prueba técnica del renderer ECS: reproduce un escenario generado por un modelo Python (dos managers, solapes con orden distinto al de slot, ocultar, deshabilitar, destruir, recorte en los bordes) y el smoke compara la VRAM de cada frame con la imagen ideal.	BIN, DSK común, CDT

`vertical` y `platformer` son las dos escenas jugables completas, ambas en ECS. Los otros veinte son pruebas técnicas deliberadamente pequeñas. `gravity`, `tilecoll`, `tilerender`, `spriteclip`, `spritemask` y `physics88` no aceptan entrada: dibujan automáticamente su resultado y fijan una paleta propia cuando la necesitan.

`platformer` usa cuatro managers ECS (monedas y puerta, jugador con el enemigo vertical izquierdo, enemigo horizontal y enemigo vertical derecho) y el renderer selectivo `video/ecs_render.s` sobre los cuatro. Al mover el jugador o retirar una moneda se activa `DIRTY`; el renderer restaura en orden inverso y redibuja por Y. Cada enemigo descuenta una vida solo al comenzar un contacto, no durante todos los frames del solapamiento. La puerta permanece oculta hasta reunir las cuatro monedas y permite pasar al segundo nivel.

El HUD ocupa una línea negra interior. Antes de redibujar texto restaura las celdas de tile que realmente cubre cada tramo: columna 3 para el dígito de monedas, columna 19 para las dos cifras de vida, columnas 0–4 para sustituir el contador completo y columnas 6–14 para retirar todo el mensaje de ayuda. Así no quedan restos de glifos anteriores ni se altera el borde.

El ejemplo ejecuta lógica y `fwk_ecs_render_prepare` antes del VSYNC; después mantiene consecutivos `fwk_ecs_render_restore` y `fwk_ecs_render_draw`, y actualiza el HUD cuando las entidades ya vuelven a estar visibles. El recorrido automatizado conduce los tramos horizontales hasta posiciones observables del jugador: el coste de render puede hacer que una iteración de juego abarque más de un frame de vídeo, por lo que no equipara un número fijo de frames del emulador con actualizaciones de lógica. El recorrido completo, el smoke focalizado de comportamiento/contactos/puerta/HUD y la prueba de texto pasan en CPC 6128. La versión focalizada inmediatamente anterior pasó también en CPC 464; la fase 0 no repite ese modelo porque no cambia contratos de memoria o modelo. No existe validación en hardware real.

La guía específica Crear un juego de plataformas con GlasnostFWK reconstruye el proyecto paso a paso, documenta la separación entre manifiesto, constantes, assets y ciclo de juego, e incluye capturas reproducibles de construcción, inicio y victoria.

9. Pruebas y emulación

9.1 Suite local

```
make test
make report
```

La suite comprueba generadores, límites de vídeo, seguridad de manifiestos, frontera de dependencias, nombres AMSDOS y colisiones del DSK común, paquete, cobertura de este manual y compilación. `make report` actualiza `docs/build-sizes.md` desde los mapas SDCC.

9.2 Smoke completo en CPC 6128

```
python3 tests/emulator_smoke.py
```

9.3 Casos específicos en CPC 464

```
python3 tests/emulator_smoke.py --model 464 resources128
python3 tests/emulator_smoke.py --model 464 diskload
python3 tests/emulator_smoke.py --model 464 audio
python3 tests/emulator_smoke.py --model 464 gravity tilecoll tilerender
spriteclip spritemask physics88
python3 tests/emulator_smoke.py --model 464 window
python3 tests/emulator_smoke.py --model 464 vertical
```

Un build correcto, una prueba host correcta, una ejecución emulada y una prueba en hardware son evidencias distintas. Consulte `docs/validation.md` para ver cuáles se han realizado realmente.

10. Crear un paquete de distribución

```
make release VERSION=0.2.0
(cd dist && sha256sum --check SHA256SUMS)
```

El empaquetador construye los ejemplos y crea en `dist/` un ZIP con los veintidós BIN, veintiún CDT y un solo `tests/GlasnostFWK-tests.dsk`, además de manifiesto, instrucciones y `SHA256SUMS`. Crear este paquete local no publica automáticamente una GitHub Release.

Use una versión con forma `X.Y.Z` y sufijo opcional válido. La release pública estable confirmada en la fecha de este manual es `v0.2.0`; los cambios locales posteriores todavía no pertenecen a esa release.

11. Diagnóstico rápido

Síntoma	Comprobación
No se encuentra CPCTelera.	Revise `CPCT_NEXT_PATH` o use `--cpct-path`; confirme que existen las herramientas y `cpctelera.lib`.
El módulo no se enlaza.	Añádalo a `modules`; para fuentes del juego use `game_sources`; para el player externo use `cpct_modules`.
Entrada siempre vacía.	Llame a `fwk_project_init`, instale una tabla persistente terminada en cero y ejecute `fwk_input_update` cada frame.
El archivo DSK no carga.	Llame a `fwk_disk_init` antes de desactivar firmware; compruebe nombre 8.3, longitud y destino.
ZX7B corrompe memoria.	Verifique últimos bytes de origen/destino y tamaños generados. No hay comprobación interna de límites.
Audio no avanza.	Inicialice música, mantenga player/datos visibles y llame a `fwk_audio_start` después del sistema.
La animación se reinicia.	No borre el estado. `fwk_ecs_anim_set` no reinicia si recibe el descriptor que la entidad ya tiene activo.
La ventana no restaura bien.	Reserve exactamente `width*height` bytes o más y no cambie descriptor, buffer ni perfil entre captura y restauración.
El cuerpo atraviesa un tile.	Compruebe tamaños/shifts, llame al resolver justo después del paso físico y mantenga el desplazamiento por eje por debajo del tamaño de tile.
El viewport devuelve carry.	Compruebe cámara, destino, dimensiones no nulas, `tile_count`, ids seleccionados y que el tileset completo no haga wrap de 16 bits.
El sprite no restaura bien.	No cambie X/Y entre captura y restauración, reserve `width*height` bytes y conserve visibles tanto los datos como el buffer.
La transparencia no compone como se espera.	Compruebe que el manifiesto declara el mismo pen que usa el fondo transparente del asset, que llamó una sola vez a `fwk_project_init` y que no sobrescribió `0x0100..0x01FF`.

Síntoma	Comprobación
Falla `make test` por el manual.	Documente el nuevo ejemplo/módulo en este HTML y ejecute `make manual`.

12. Mantener y ampliar este manual

Este documento forma parte del contrato del proyecto. Siempre que se añada, cambie o retire una funcionalidad:

1. Actualice la explicación y el contrato correspondiente en `docs/manual.html`.
2. Si aparece un módulo seleccionable, marque su sección con `data-module="ruta/del/modulo.s"`.
3. Si aparece un ejemplo, añada una fila con `data-example="nombre"`.
4. Regenerar PDF y checksum: `make manual`.
5. Durante la edición use `make check CHECK_EXAMPLES="ejemplos afectados"`. Al cerrar el cambio, ejecute `make test` una sola vez y registre el resultado real en `PROJECT_STATUS.md`.

```
python3 -m venv .venv-docs
.venv-docs/bin/python -m pip install -r docs/requirements-manual.txt
make manual PYTHON=.venv-docs/bin/python
make test
```

La prueba `tests/test_manual.py` compara los ejemplos y módulos seleccionados con las marcas del manual, valida que el PDF existe y comprueba que `manual.sha256` corresponde al HTML actual. Así, un cambio funcional obliga a revisar esta documentación antes de integrar.

Fuentes complementarias: `docs/architecture-and-api.md`, `docs/validation.md`, `docs/provenance.md` y `PROJECT_STATUS.md`. Ante discrepancias, los contratos del código y la evidencia reproducible prevalecen sobre una descripción antigua.